DEVELOPER REFERENCE

Generated 12 October 2026 · package version v1.7.1

Using `peoplelogy/laravel-xero-bridge` — MyInvois TIN validation

A practical reference for checking a Malaysian taxpayer's TIN against LHDN (HASiL) through the package's MyInvois module: turning it on, the one call, what its answers mean, what goes wrong, and the two optional records it can keep of every check.

This is one of the usage guides in this folder, written for developers. **Xero usage** covers the Xero side; **Invoice usage** shows where a validated TIN and BRN go on the contact before an invoice is sent. Staff who only run checks from the test console want the **MyInvois user manual** instead.

If you are not in Malaysia, this guide does not apply to you. The module ships off. It registers no route, needs no configuration, and changes nothing about how the rest of the package behaves.

> **Version note.** Written against `peoplelogy/laravel-xero-bridge` v1.7.1. Method signatures are read from the package source; the contract is documented in full in `08-myinvois-tin-validation` and `09-persistence` in the folder above this one, which are the authority wherever the two disagree. TINs and registration numbers in the samples are illustrative.

Contents

1. What it does
2. Turning it on
3. Validating a TIN
4. What the answer means
5. Errors
6. From the test console
7. Recording the checks
8. Caching and rate limits
9. Troubleshooting

What it does

One LHDN endpoint, **Validate Taxpayer's TIN**, and one boolean back: is this TIN genuinely paired with this identifier in HASiL's records?

```
GET {host}/api/v1.0/taxpayer/validate/{tin}?idType={idType}&idValue={idValue}
```

The TIN and BRN it checks are the same two values the Xero side writes onto a contact as `TaxNumber` and `CompanyNumber`, which is why the module lives in this package. It shares no code with the Xero side: it has its own config file (`config/myinvois.php`), its own namespace (`Peoplelogy\XeroBridge\MyInvois\`) and its own exception.

Turning it on

```
MYINVOIS_ENABLED=true
MYINVOIS_ENVIRONMENT=production       # or sandbox
MYINVOIS_CLIENT_ID=...
MYINVOIS_CLIENT_SECRET=...
```

then `php artisan config:clear` if your configuration is cached. Publishing the annotated config is optional — `php artisan vendor:publish --tag=myinvois-config` — and a separate tag from the Xero config, so republishing one can never overwrite the other.

> **Sandbox and production issue SEPARATE credentials**
>
> The Client ID and secret come from the MyInvois portal, under the taxpayer's ERP registration — `preprod.myinvois.hasil.gov.my` for sandbox, `myinvois.hasil.gov.my` for production. A Client ID from one is **always** rejected by the other with `invalid_client`. So `MYINVOIS_ENVIRONMENT` must name the portal the credentials came from; changing it without changing the credentials is a misconfiguration, not a promotion. Production credentials checked against production are safe to use from any environment of yours, local included: validation only reads.

The client refuses to send an obvious placeholder (`0`, `YOUR_CLIENT_ID`, `changeme`…) — LHDN **auto-blocks** a Client ID that sends one — and names the key instead.

Validating a TIN

```
use Peoplelogy\XeroBridge\MyInvois\Facades\MyInvois;
use Peoplelogy\XeroBridge\MyInvois\IdType;

$valid = MyInvois::validate('C21264688050', IdType::BRN, '201001040438');
```

Or inject the client, which suits a service class and a test:

```
use Peoplelogy\XeroBridge\MyInvois\MyInvoisClient;

public function __construct(private readonly MyInvoisClient $myInvois) {}
```

Parameters

Argument	Type	Notes
`$tin`	`string`	The taxpayer's TIN. No format check is applied — LHDN publishes none, and TINs carry letter prefixes that vary by taxpayer class.
`$idType`	`IdType|string`	`NRIC`, `PASSPORT`, `BRN` or `ARMY`, any casing. Anything else throws before a request is made.
`$idValue`	`string`	The identifier of that type: the 12-digit SSM registration number for `BRN`, the IC number for `NRIC`.

`IdType`	What it is
`BRN`	Business registration number (SSM) — a company
`NRIC`	Malaysian identity card number — a person
`PASSPORT`	Passport number — a person
`ARMY`	Army number — a person

Naming the record it is about

```
MyInvois::for($customer)->validate($tin, IdType::BRN, $brn);
```

`for()` returns a copy carrying the record, so the optional records below can be found — and erased — from your customer later. It changes nothing about the call itself.

Also available

```
MyInvois::lastCorrelationId();   // LHDN's id for the last call — what their support asks
for
MyInvois::lastRateLimit();       // remaining / limit / reset, as LHDN reported them
MyInvois::forgetToken();         // drop the cached access token, e.g. after rotating the
secret
```

What the answer means

Result	LHDN said	Meaning
`true`	HTTP 200	HASiL holds this TIN **paired with** this identifier
`false`	HTTP 404	It does not. This is an **answer**, not a failure, so it is not thrown

- **Both values are required, and that is the point.** Since **1 August 2026** LHDN validates the TIN and the identifier *as a pair*. A real TIN with a stale or mistyped BRN answers `false` exactly like a fabricated one — so a `false` is usually the BRN, not the TIN.
- **A match is narrower than it looks.** It proves the pair exists in HASiL's records. It does **not** prove the pair belongs to the customer you are invoicing: the endpoint returns no name, no address, no body at all. Treat it as "this is a real TIN/ID pair", never as identity verification.

Errors

Everything that is not an answer throws `Peoplelogy\XeroBridge\MyInvois\MyInvoisException`. It is deliberately **not** a `XeroBridgeException`, so code catching Xero failures cannot swallow an LHDN one by accident.

```
use Peoplelogy\XeroBridge\MyInvois\MyInvoisException;

try {
    $valid = MyInvois::validate($tin, IdType::BRN, $brn);
} catch (MyInvoisException $e) {
    if ($e->isConfigurationProblem()) {
        // A person must fix a setting or the portal registration. Do NOT retry.
    } elseif ($e->isRetryable()) {
        // Rate limit or an LHDN outage: back off, $e->retryAfter() when present.
    }

    Log::error('MyInvois validation failed', $e->context());
}
```

Situation	What you get
Pair exists	`true`
Pair not found (404)	`false`
Module off, a setting blank, or a placeholder	throws, `isConfigurationProblem()` true, the message names the key
Credentials rejected (`invalid_client`)	throws, `isConfigurationProblem()` true — usually the wrong `MYINVOIS_ENVIRONMENT`
Not entitled to the API (403)	throws, `isConfigurationProblem()` true — a portal registration matter
Malformed request (400)	throws, not retryable
Rate limited (429)	throws, `isRetryable()` true, `retryAfter()` set
LHDN down (500/503)	throws, `isRetryable()` true

Accessors carry LHDN's own error envelope: `statusCode()`, `errorCode()`, `errorMalay()`, `propertyName()`, `propertyPath()`, `target()`, `innerErrors()`, `retryAfter()` and `correlationId()`. `context()` is everything safe to log — never a token or the secret.

From the test console

With the module on, the test console at `/xero/console` — itself off until `XERO_CONSOLE_ENABLED=true` — shows an **LHDN MyInvois** panel with two actions:

Action	What it does
`myinvois.validate`	Runs one validation from a TIN, an id type and an id value, and shows the answer, the timing and LHDN's rate-limit headers
`myinvois.forget_token`	Drops the cached access token

The panel goes through the same client as your code, so its checks are recorded like any other (below). Behind the default `web,auth` middleware any signed-in user can reach the console; narrow `XERO_CONSOLE_MIDDLEWARE` — for example to `web,auth,role:admin|super_admin` — wherever you turn it on. The grey numbers in its fields are placeholders, not values: type the real ones.

Recording the checks

Nothing about a check is stored unless you switch it on. Two records are available, each in a table of its own; use either or both.

	Verdict record	Check history
Switch	`MYINVOIS_AUDIT=true`	`MYINVOIS_HISTORY=true`
Table	`myinvois_validations`	`myinvois_validation_history`
Rows	**One per TIN/ID pair** — a re-check updates it (`check_count`, `last_checked_at`)	**One per check**, never updated
Answers kept	Match and no-match	Match, no-match **and failures** (credentials refused, rate limit, outage, timeout, a setting left blank)
Who ran it	—	The signed-in user (`actor_type`, `actor_id`, `actor_guard`)
The numbers	Never — a keyed hash and the TIN's last four only	**BRN check:** the full TIN and BRN. **NRIC / PASSPORT / ARMY check:** neither, not even the last four
Kept for	`MYINVOIS_AUDIT_RETAIN_DAYS`, 400 days after the last check	`MYINVOIS_HISTORY_RETAIN_DAYS`, 400 days after the check

Both also keep the id type, the environment, LHDN's correlation id, the rules version, and the record named with `MyInvois::for()`. Both are filed under the same keyed `subject_hash`, so the two tables join.

Switching them on

```
php artisan vendor:publish --tag=myinvois-migrations   # adds only what you do not have yet
php artisan migrate
```

```
MYINVOIS_AUDIT=true      # the latest verdict per pair
MYINVOIS_HISTORY=true    # every check
```

```
// routes/console.php -- keeps both tables to their retention
Schedule::command('xero-bridge:prune')->daily();
```

A queue worker that started before the migration keeps believing the table is missing until it restarts: run `php artisan queue:restart` after migrating.

Why a person's numbers are never kept

A company's BRN is public SSM data and is what you match against your own customer record, so a BRN check's history row keeps the full TIN and BRN. An NRIC, passport or army number identifies a person, and so does that person's TIN — personal data under Malaysia's PDPA — so that row keeps neither. It is still written, with its outcome, who ran it and when, and the numbers are replaced with `[redacted]` in any stored error message too, since a connection error quotes the URL that carried them. There is no setting to change this.

Reading them

```
use Peoplelogy\XeroBridge\MyInvois\IdType;
use Peoplelogy\XeroBridge\MyInvois\Models\MyInvoisValidation;
use Peoplelogy\XeroBridge\MyInvois\Models\MyInvoisValidationHistory;
use Peoplelogy\XeroBridge\MyInvois\MyInvoisHistory;

// Every check of one pair, newest first -- by the keyed hash, so a person's checks are
found too.
$hash = app(MyInvoisHistory::class)->subjectHash($tin, IdType::NRIC, $nric);
MyInvoisValidationHistory::forSubject($hash)->latest('id')->get();

// Everything checked for one of your customers.
MyInvoisValidationHistory::forOwner($customer->getMorphClass(), (string) $customer-
>getKey())->get();

// What failed today.
MyInvoisValidationHistory::failed()->whereDate('created_at', today())->get();

// The latest verdict for a customer.
MyInvoisValidation::forOwner($customer->getMorphClass(), (string) $customer->getKey())-
>first();
```

`matched()` and `unmatched()` scopes exist on the history too. A verdict recorded under older LHDN rules can be spotted with `$verdict->isStale(MyInvoisAudit::CURRENT_RULES_VERSION)`.

Erasure

```
use Peoplelogy\XeroBridge\MyInvois\MyInvoisAudit;
use Peoplelogy\XeroBridge\MyInvois\MyInvoisHistory;

// From the customer model's deleting event:
app(MyInvoisAudit::class)->forgetOwner($customer);
app(MyInvoisHistory::class)->forgetOwner($customer);

// When you still hold the numbers:
app(MyInvoisAudit::class)->forgetSubject($tin, $idType, $idValue);
app(MyInvoisHistory::class)->forgetSubject($tin, $idType, $idValue);
```

The hash is keyed from your application key, so rotating `APP_KEY` makes earlier rows unmatchable by subject — the next check simply writes new ones.

Recording never breaks a check: a row that cannot be written is logged as a warning and the answer is returned as usual.

Caching and rate limits

Limit (per Client ID)	Applies to	What the package does
12 per minute	Token requests	Caches the access token — mandatory, not an optimisation — for its hour less `MYINVOIS_TOKEN_LEEWAY` (60 s). Sandbox and production tokens are cached apart.
60 per minute	Validations	Nothing by default. LHDN calls exceeding it "malicious activity", so a bulk sweep over a customer table needs pacing.

Validation **results** are not cached unless you choose to:

```
MYINVOIS_RESULT_MATCHED_TTL=86400    # keep a "match" for a day
MYINVOIS_RESULT_UNMATCHED_TTL=300    # keep a "no match" for five minutes, or 0
```

The two are asymmetric on purpose: a match is a durable fact, a no-match is usually a typo about to be corrected. Keep a match short too — a company can be deregistered between your cache write and your invoice. Cache keys hash the TIN and identifier, so neither sits in plain text in the store. An answer served from the cache is still recorded in the check history, marked `from_cache`.

Troubleshooting

"MyInvois refused the credentials (invalid_client)." The Client ID and secret do not match, or — most often — they belong to the other environment. The message names the environment it tried; set `MYINVOIS_ENVIRONMENT` to the portal the credentials came from, run `php artisan config:clear` where config is cached, and try **once**. LHDN can block a Client ID that keeps sending rejected credentials, so do not loop on it.

"MyInvois is disabled." `MYINVOIS_ENABLED=true`, then `php artisan config:clear`.

"MyInvois [client_id] is still set to a placeholder." Put the real value in. The check is local, made before anything is sent, precisely so a placeholder never reaches LHDN.

HTTP 403. The credentials are valid but not entitled to the taxpayer validation API — a portal registration matter, not a code one. Do not retry it.

Everything valid returns `false`. Almost always the BRN rather than the TIN. Since 1 August 2026 the pair is validated together, so a stale registration number — or one still in the pre-October-2019 short format — fails even with a perfect TIN. Ask the buyer for their current 12-digit SSM number.

The verdict or history table stays empty. Its switch is off (`MYINVOIS_AUDIT` / `MYINVOIS_HISTORY`), its migration was never published and run, or a queue worker started before the migration and needs `php artisan queue:restart`.

cURL error 60, "self signed certificate in certificate chain". The package offers no TLS-verification switch. Point php.ini's `curl.cainfo` and `openssl.cafile` at a current `cacert.pem` — usually the whole problem on a stock MAMP, XAMPP or Windows PHP — or, behind a TLS-inspecting proxy, add that proxy's root CA to the system trust store.